



Autenticação via Token com JSON Web Tokens (JWT)

Nossas aplicações até o momento tem todos os métodos programados com acesso livre na controller, mas podemos fazer com que os métodos das controllers sejam acessados apenas se houver uma autenticação válida, porém, esse processo acarretaria a necessidade de entrar com as credenciais a cada método solicitado.

Para facilitar os procedimentos, podemos utilizar um token que pode ficar armazenado no navegador de uma aplicação ou de um dispositivo, sendo que este token conterá informações essenciais de identificação de um usuário, mas não informações sensíveis que podem ser roubadas, além disso, o token pode ser criado já com uma data de validade atribuída.

JSON Web Tokens é um recurso muito útil para tudo isso que falamos até então e faremos a aplicação prática na API, mas para que conheça um pouco da teoria do que estamos falando, segue um vídeo para introdução sobre Token e JWT:

Token: <https://youtu.be/LtVb9rhU41c>

JWT: <https://youtu.be/Gyq-yeot8qM>

Artigo: <https://www.brunobrito.net.br/jwt-cookies-oauth-bearer/>

O JWT nada mais é do que uma série de caracteres que contém especificações sobre um usuário, chamamos essas especificações de Claims (Reivindicações), ou seja, através desses dados é possível saber quais os tipos de acesso que determinado usuário poderá ter numa API, por exemplo, ou simplesmente resgar informações armazenadas no token gerado.

1. Abra o projeto de API, vá até o arquivo appsettings.json e insira a codificação que será a chave para gerar o token

```
{
  "ConfiguracaoToken": {
    "Chave": "minha chave super secreta minha chave super secreta minha chave super secreta"
  },
  "ConnectionStrings": {
    "ConexaoLocal": "Data Source=localhost; Initial Catalog=DB-DS-DS_2023_2; User Id=sa; Pass"
  }
}
```

2. Utilize o terminal para instalar os pacotes necessários para programação do método que vai gerar o Token:
 - ➔ dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
 - ➔ dotnet add package System.IdentityModel.Tokens.Jwt
 - ➔ dotnet add package Microsoft.IdentityModel.Tokens

Perceba que ao abrir o arquivo ApiLibertadoresCopa.csProj, as referências aos pacotes estarão dentro da tag **ItemGroup** conforme o exemplo. A versão poderá estar mais avançada.

```
<PackageReference Include="Microsoft.IdentityModel.Tokens" Version="8.22.0"/>
<PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="8.22.0"/>
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.11"/>
```



3. Confirme que a classe `Usuario` já tem uma propriedade chamada `Token` e com `NotMapped`, conforme abaixo

```
[NotMapped]
0 references
public string Token { get; set; } = string.Empty;
```

4. Altere o construtor da classe **UsuariosController** para permitir acesso as configurações criadas anteriormente. Utilize o using *Microsoft.Extensions.Configuration*.

```
private readonly DataContext _context;
1 reference
private readonly IConfiguration _configuration;
0 references
public UsuariosController(DataContext context, IConfiguration configuration)
{
    _context = context;
    _configuration = configuration;
}
```

5. Na classe `UsuariosController` desenvolva o método que criará o token. Usings: `System.Security.Claims`, `Microsoft.IdentityModel.Tokens`, `System.Text`, `System.IdentityModel.Tokens.Jwt`

```
private string CriarToken(Usuario usuario)
{
    List<Claim> claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
        new Claim(ClaimTypes.Name, usuario.Username)
    };
    SymmetricSecurityKey key = new SymmetricSecurityKey(Encoding.UTF8
        .GetBytes(_configuration.GetSection("ConfiguracaoToken:Chave").Value));
    SigningCredentials creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha5
12Signature);
    SecurityTokenDescriptor tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(claims),
        Expires = DateTime.Now.AddDays(1),
        SigningCredentials = creds
    };
    JwtSecurityTokenHandler tokenHandler = new JwtSecurityTokenHandler();
    SecurityToken token = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(token);
}
```



6. Altere o retorno do método de autenticação para que no return Ok, usemos o trecho abaixo.

```
else
{
    usuario.PasswordString = string.Empty;
    usuario.PasswordHash = null;
    usuario.PasswordSalt = null;
    usuario.Token = CriarToken(usuario);
    return Ok(usuario);
}
```

7. Abra o postman e realize o teste de Autenticação de um usuário já salvo na base, confirmando que será exibido o token, caso a autenticação tenha sucesso. Você pode conferir os dados existentes no Token copiando-o e usando no site <https://jwt.io/>
8. Abra a classe Program.cs e faça a edição do serviço AddAuthentication como segue abaixo. Esta etapa será importante para que possamos resgatar as informações do Token. Necessário using para Microsoft.AspNetCore.Authentication.JwtBearer; Microsoft.IdentityModel.Tokens e System.Text.

```
builder.Services.AddControllers().AddJsonOptions(options =>
{
    options.JsonSerializerOptions.ReferenceHandler =
        System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
});

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.ASCII
                .GetBytes(builder.Configuration.GetSection("ConfiguracaoToken:Chave").Value)),
            ValidateIssuer = false,
            ValidateAudience = false
        };
    });

var app = builder.Build();
```



9. Ainda na classe Program.cs, adicione a linha que implanta a Autenticação e Autorização como a seguir

```
app.UseAuthentication();  
app.UseAuthorization();  
  
app.MapControllers();  
app.Run();
```

10. Vá até a controller de Usuários e adicione no topo da classe o atributo de Autorização. Com o atributo Authorize iremos limitar quem pode acessar a controller ou algum método dentro da Controller. Iniciaremos realizando os procedimentos de configuração.

```
[Authorize]  
[ApiController]  
[Route("[controller]")]  
0 references  
public class UsuariosController : ControllerBase  
{  
    5 references  
    private readonly DataContext _context;
```

- Será necessário o *using Microsoft.AspNetCore.Authorization*.
- Tente realizar o teste do método de autenticação no postman para verificar qual status é retornado.

11. Provavelmente, o teste da etapa anterior deve ter retornado uma mensagem 401 (Unauthorized). Vamos inserir uma permissão para apenas para os métodos “**Autenticar**” e “**Registrar**” conforme a seguir

```
[AllowAnonymous]  
[HttpPost("Autenticar")]  
0 references  
public async Task<IActionResult> AutenticarUsuario(Usuario credenciaisUsuario)  
{  
    Usuario usuario = await _context.Usuarios.FirstOrDefaultAsync(x =>  
        x.Username.ToLower().Equals(credenciaisUsuario.Username.ToLower()));
```

- Esse atributo concede uma exceção para que um usuário anônimo consiga acessar o método contido numa controller restrita.
- Realize novamente o teste e copie o token gerado e guarde em um bloco de notas para usarmos, quando necessário.

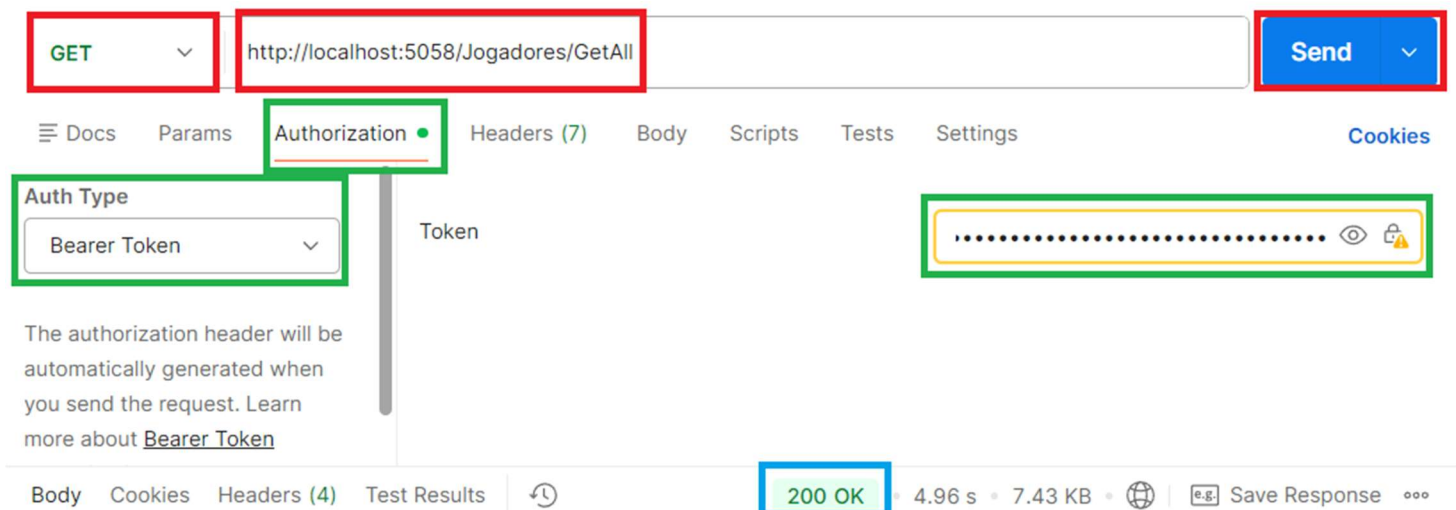


12. Adicione o atributo Authorize no topo da controller de Jogadores também

```
[Authorize]
[ApiController]
[Route("[controller]")]
1 reference
public class JogadoresController : ControllerBase
{
```

- Será necessário o `using Microsoft.AspNetCore.Authorization`

13. Configuraremos o postman para testar o Método GetAll de Jogadores passando os dados do Token no cabeçalho da requisição http, conforme as configurações feitas na sinalização em verde. A sinalização em Azul é o resultado obtido e a sinalização em vermelho são as configurações gerais que usamos para métodos get.



- Perceba que adicionamos a key (chave) “Authorization” e no Type a palavra “Bearer” (indica que é uma autorização de token Jwt). Dessa maneira, a API reconhecerá que existe permissão para acessar os métodos da controller.



Identificação centralizada do Usuário

Ao invés de criar a codificação para identificar o usuário em cada método, criaremos a seguir uma classe e métodos para coletar os dados do usuário através do token que poderá ser usado nos demais métodos da controllers, e caso tenhamos que usar em outras controllers, será desenvolvido da mesma maneira.

1. Crie uma pasta chamada **Extensions** e dentro da pasta crie uma classe estática chamada **ClaimTypesExtesion**. Uma classe estática é aquela pode possuir métodos estáticos, sem que tenhamos que criar um objeto instanciado.

```
public static class ClaimTypesExtension
```

2. Crie um método de extensão conforme abaixo. Ele retornará o Id do usuário de acordo com a leitura feita na claim que está dentro do token. Necessário **using System.Security.Claims**.

```
public static int UsuarioId(this ClaimsPrincipal user)
{
    try
    {
        var usuarioId = user.Claims.FirstOrDefault(x => x.Type == ClaimTypes.NameIdentifier)?.Value ?? string.Empty;
        return int.Parse(usuarioId);
    }
    catch
    {
        return 0;
    }
}
```



Autenticação baseada em Papéis

Criaremos definições de perfis ou papéis para cada usuário, e assim, resgatar qual o tipo de usuário está requisitando operações para apresentar os dados e acessos que ele pode ter. Além disso, com a esta programação da API sendo realizada, será possível programar o aplicativo para aplicar as chamadas da API.

Acrescentando mais uma claim para o Token:

Até este momento, guardávamos na coleção de claims os Id o usuário e o login dele, faremos a programação para acrescentar uma claim para guardar Perfil.

1. Abra a controller de usuário e modifique o método CriarToken

```
private string CriarToken(Usuario usuario)
{
    List<Claim> claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
        new Claim(ClaimTypes.Name, usuario.Username),
        new Claim(ClaimTypes.Role, usuario.Perfil)
    };
}
```

2. Execute a API, faça o teste no postman. Copie o token gerado e use o site <https://jwt.io/> para ver o conteúdo da Claim, devendo constar o Perfil.
3. Abra a controller de Jogador e altere a anotação Authorize, que indicará o tipo de usuário que poderá acessar a controller.

```
[Authorize(Roles="UsuarioComum")]
```

4. Registre um usuário que você mais usa para o perfil Admin. Execute a API, autentique e tente buscar todos os jogadores

Id	Username	PasswordHash	PasswordSalt	Perfil
1	UsuarioTeste	0x362EF2FA76A574F1...	0xA027E77...	UsuarioComum
2	UsuarioAdmin	0xD140DF4E84D64B7...	0x4C9834A...	Admin

5. Abra o postman, faça a autenticação e use o Token para obter todos os jogadores.

```
GET http://localhost:5058/Jogadores/GetAll
```

6. Você perceberá que vai dar o erro 403, de proibição. Adicione a palavra Admin nas permissões da controller, execute a API e teste novamente.

```
[Authorize(Roles="UsuarioComum,Admin")]
```